Overview of Gradient descent

Subject: Gradient descent

$$\theta_{t+1} = \theta_t - \eta \nabla_\theta L(\theta_t)$$

1.

f (x (1)) ≤ f (x (0)) = f (0) . {\displaystyle f\left(\mathbf {x} ^{(1)}\right)\leq f\left(\mathbf {x} ^{(0)}\right)=f(\mathbf {0}).}

2.

The line search minimization, finding the locally optimal step size η {\displaystyle \eta } on every iteration, can be performed analytically for quadratic functions, and explicit formulas for the locally optimal η {\displaystyle \eta } are known. For example, for real symmetric and positive-definite matrix A {\displaystyle \mathbf {A} } , a simple algorithm can be as follows,

3.

x (1) = 0 − η 0 ∇ f (0) = 0 − η 0 J G (0) ■ G (0) , {\displaystyle \mathbf {x} ^{(1)}=\mathbf {0} -\eta _{0}\nabla f(\mathbf {0})=\mathbf {0} -\eta _{0}J_{G}(\mathbf {0})^{\top }G(\mathbf {0}),}

4.

Choosing the step size and descent direction Since using a step size η {\displaystyle \eta } that is too small would slow convergence, and a η {\displaystyle \eta } too large would lead to overshoot and divergence, finding a good setting of η {\displaystyle \eta } is an important practical problem. Philip Wolfe also advocated using "clever choices of the [descent] direction" in practice. While using a direction that deviates from the steepest descent direction may seem counter-intuitive, the idea is that the smaller slope may be compensated for by being sustained over a much longer distance. To reason about this mathematically, consider a direction p n {\displaystyle \mathbf {p} _{n}} and step size η n {\displaystyle \eta _{n}} and consider the more general update:

5.

∇ f (x) = 2 A ■ (A x − b) . {\displaystyle \nabla f(\mathbf {x})=2\mathbf {A} ^{\top }(\mathbf {A} \mathbf {x} -\mathbf {b}).}

6.

Gradient descent is based on the observation that if the multi-variable function f (x) {\displaystyle f(\mathbf {x})} is defined and differentiable in a neighborhood of a point a {\displaystyle \mathbf {a} } , then f (x) {\displaystyle f(\mathbf {x})} decreases fastest if one goes from a {\displaystyle \mathbf {a} } in the direction of the negative gradient of f {\displaystyle f} at a , i . e . , − ∇ f (a) {\displaystyle \mathbf {a} ,i.e.,-\nabla f(\mathbf {a})} . It follows that, if

7.

(exp ■ (− x 1 x 2) + 20 x 3 + 10 π − 3 3) 2] , {\displaystyle {\begin{aligned}f(\mathbf {x})&={\frac {1}{2}}G^{\top }(\mathbf {x})G(\mathbf {x})\\&={\frac {1}{2}}\left[\left(3x_{1}-\cos(x_{2}x_{3})-{\frac {3}{2}}\right)^{2}+\left(4x_{1}^{2}-625x_{2}^{2}+2x_{2}-1\right)^{2}+\right.\\&{}\qquad \left.\left(\exp(-x_{1}x_{2})+20x_{3}+{\frac {10\pi -3}{3}}\right)^{2}\right],\end{aligned}}}

8.

Further reading Boyd, Stephen; Vandenberghe, Lieven (2004). "Unconstrained Minimization" (PDF). Convex Optimization. New York: Cambridge University Press. pp. 457–520. ISBN 0-521-83378-7. Chong, Edwin K. P.; ■ak, Stanislaw H. (2013). "Gradient Methods". An Introduction to Optimization (Fourth ed.). Hoboken: Wiley. pp. 131–160. ISBN 978-1-118-27901-4. Himmelblau, David M. (1972). "Unconstrained Minimization Procedures Using Derivatives". Applied Nonlinear Programming. New York: McGraw-Hill. pp. 63–132. ISBN 0-07-028921-2.

9.

Comments Gradient descent works in spaces of any number of dimensions, even in infinite-dimensional ones. In the latter case, the search space is typically a function space, and one calculates the Fréchet derivative of the functional to be minimized to determine the descent direction. That gradient descent works in any number of dimensions (finite number at least) can be seen as a consequence of the Cauchy–Schwarz inequality, i.e. the magnitude of the inner (dot) product of two vectors of any dimension is maximized when they are colinear. In the case of gradient descent, that would be when the vector of independent variable adjustments is proportional to the gradient vector of partial derivatives. The gradient descent can take many iterations to compute a local minimum with a required accuracy, if the curvature in different directions is very different for the given function. For such functions, preconditioning, which changes the geometry of the space to shape the function level sets like concentric circles, cures the slow convergence. Constructing and applying preconditioning can be computationally expensive, however. The gradient descent can be modified via momentums (Nesterov, Polyak, and Frank–Wolfe) and heavy-ball parameters (exponential moving averages and positive-negative momentum). The main examples of such optimizers are Adam, DiffGrad, Yogi, AdaBelief, etc. Methods based on Newton's method and inversion of the Hessian using conjugate gradient techniques can be better alternatives. Generally, such methods converge in fewer iterations, but the cost of each iteration is higher. An example is the BFGS method which consists in calculating on every step a matrix by which the gradient vector is multiplied to go into a "better" direction, combined with a more sophisticated line search algorithm, to find the "best" value of η . {\displaystyle \eta .} For extremely large problems, where the computer-memory issues dominate, a limited-memory method such as L-BFGS should be used instead of BFGS or the steepest descent. While it is sometimes possible to substitute gradient descent for a local search algorithm, gradient descent is not in the same family: although it is an iterative method for local optimization, it relies on an objective function's gradient rather than an explicit exploration of a solution space. Gradient descent can be viewed as applying Euler's method for solving ordinary differential equations x ′ (t) = − ∇ f (x (t)) {\displaystyle x'(t)=-\nabla f(x(t))} to a gradient flow. In turn, this equation may be derived as an optimal controller for the control system x ′ (t) = u (t) {\displaystyle x'(t)=u(t)} with u (t) {\displaystyle u(t)} given in feedback form u (t) = − ∇ f (x (t)) {\displaystyle u(t)=-\nabla f(x(t))} .

10.

Because each step is taken in the steepest direction, steepest-descent steps alternate between directions aligned with the extreme axes of the elongated level sets. When κ (A) {\displaystyle \kappa (\mathbf {A})} is large, this produces a characteristic zig–zag path. The poor conditioning of A {\displaystyle \mathbf {A} } is the primary cause of the slow convergence, and orthogonality of successive residuals reinforces this alternation. As shown in the image on the right, steepest descent converges slowly due to the high condition number of A {\displaystyle \mathbf {A} } , and the orthogonality of residuals forces each new direction to undo the overshoot from the previous step. The result is a path that zigzags toward the solution. This inefficiency is one reason conjugate gradient or preconditioning methods are preferred.